

第 4 讲：值迭代与策略迭代

第 4 讲：值迭代与策略迭代

在前两讲的基础上，我们现在可以介绍第一批能够求解最优策略的算法。本章将介绍三个密切相关的算法：**值迭代 (Value Iteration)**、**策略迭代 (Policy Iteration)** 和 **截断策略迭代 (Truncated Policy Iteration)**。值迭代正是第 3 讲中压缩映射定理所建议的求解贝尔曼最优方程的算法；策略迭代的基本思想在强化学习中得到了广泛应用；截断策略迭代则是统一前两者的一个更一般的算法框架。

本章介绍的算法被称为**动态规划 (Dynamic Programming)** 算法，它们都需要已知系统模型（即 $p(r | s, a)$ 和 $p(s' | s, a)$ ）。这些算法是后续章节无模型强化学习算法的重要基础：例如，第 5 讲介绍的蒙特卡洛方法可以通过推广本章的策略迭代算法直接得到。

4.1 值迭代

值迭代正是第 3 讲中压缩映射定理所建议的求解贝尔曼最优方程的算法。具体而言，该算法的矩阵-向量形式为：

$$\mathbf{v}_{k+1} = \max_{\pi \in \Pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_k), \quad k = 0, 1, 2, \dots$$

由定理 3.3 保证，当 $k \rightarrow \infty$ 时， $\mathbf{v}_k$ 收敛到最优状态价值， π_k 收敛到最优策略。

该算法是迭代的，每次迭代包含两个步骤：

- **策略更新步骤 (Policy Update)**：求解以下优化问题得到策略

$$\pi_{k+1} = \arg \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_k)$$

其中 $\mathbf{v}_k$ 来自上一次迭代。

- **价值更新步骤 (Value Update)**：计算新的价值

$$\mathbf{v}_{k+1} = \mathbf{r}_{\pi_{k+1}} + \gamma \mathbf{P}_{\pi_{k+1}} \mathbf{v}_k \tag{1}$$

其中 $\mathbf{v}_{k+1}$ 将在下一次迭代中使用。

上述矩阵-向量形式有助于理解算法的核心思想，但要实现该算法，还需要进一步分析其元素形式。

4.1.1 元素形式与实现

考虑第 k 步迭代和状态 s 。

策略更新步骤的元素形式： $\pi_{k+1} = \arg \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_k)$ 展开为

$$\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a | s) \underbrace{\left(\sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_k(s') \right)}_{q_k(s, a)}, \quad s \in \mathcal{S}$$

由第 3.3.1 节的结论，求解该最大化问题的最优策略为贪心策略：

$$\pi_{k+1}(a | s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & a \neq a_k^*(s) \end{cases} \quad (2)$$

其中 $a_k^*(s) = \arg \max_a q_k(s, a)$ 。若 $\arg \max_a q_k(s, a)$ 有多个解，可任意选择其中一个而不影响算法的收敛性。

价值更新步骤的元素形式： $v_{k+1} = \mathbf{r}_{\pi_{k+1}} + \gamma \mathbf{P}_{\pi_{k+1}} v_k$ 展开为

$$v_{k+1}(s) = \sum_a \pi_{k+1}(a | s) \underbrace{\left(\sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_k(s') \right)}_{q_k(s, a)}, \quad s \in \mathcal{S}$$

将 (2) 代入上式可得：

$$v_{k+1}(s) = \max_a q_k(s, a)$$

综上，值迭代的核流程可以概括为：

$$v_k(s) \rightarrow q_k(s, a) \rightarrow \text{新的贪心策略 } \pi_{k+1}(s) \rightarrow \text{新的价值 } v_{k+1}(s) = \max_a q_k(s, a)$$

算法 4.1 值迭代算法

初始化： 已知所有 (s, a) 的概率模型 $p(r | s, a)$ 和 $p(s' | s, a)$ 。初始猜测 v_0 。

目标： 求解贝尔曼最优方程，得到最优状态价值与最优策略。

当 v_k 未收敛（即 $\|v_k - v_{k-1}\|$ 大于预设阈值），对第 k 次迭代执行：

- 对每个状态 $s \in \mathcal{S}$ ，执行：
 - 对每个动作 $a \in \mathcal{A}(s)$ ，计算： $q_k(s, a) = \sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_k(s')$
 - 最大动作价值： $a_k^*(s) = \arg \max_a q_k(s, a)$
 - 策略更新： $\pi_{k+1}(a | s) = 1$ 若 $a = a_k^*$ ，否则为 0
 - 价值更新： $v_{k+1}(s) = \max_a q_k(s, a)$

备注

一个可能令人困惑的问题是： v_k 是否是一个状态价值？答案是否。虽然 v_k 最终收敛到最优状态价值，但它并不保证满足任何策略的贝尔曼方程。例如，它通常不满足 $v_k = \mathbf{r}_{\pi_{k+1}} + \gamma \mathbf{P}_{\pi_{k+1}} v_k$ 或 $v_k = \mathbf{r}_{\pi_k} + \gamma \mathbf{P}_{\pi_k} v_k$ 。 v_k 仅仅是算法产生的中间值。同理， q_k 也不是动作价值。

4.1.2 示例

我们通过一个简单的 2×2 网格世界来说明值迭代算法的逐步执行过程。该示例中 s_4 为目标区域，奖励设置 $r_{\text{boundary}} = r_{\text{forbidden}} = -1$ ， $r_{\text{target}} = 1$ ，折扣因子 $\gamma = 0.9$ 。每个状态-动作对的 q 值表达式如表 4.1 所示。

表 4.1 示例的 $q(s, a)$ 表达式

$q\text{-table}$	a_1	a_2	a_3	a_4	a_5
s_1	$-1 + \gamma v(s_1)$	$-1 + \gamma v(s_2)$	$0 + \gamma v(s_3)$	$-1 + \gamma v(s_1)$	$0 + \gamma v(s_1)$
s_2	$-1 + \gamma v(s_2)$	$-1 + \gamma v(s_2)$	$1 + \gamma v(s_4)$	$0 + \gamma v(s_1)$	$-1 + \gamma v(s_2)$
s_3	$0 + \gamma v(s_1)$	$1 + \gamma v(s_4)$	$-1 + \gamma v(s_3)$	$-1 + \gamma v(s_3)$	$0 + \gamma v(s_3)$
s_4	$-1 + \gamma v(s_2)$	$-1 + \gamma v(s_4)$	$-1 + \gamma v(s_4)$	$0 + \gamma v(s_3)$	$1 + \gamma v(s_4)$

$k = 0$: 不妨设初始价值 $v_0(s_1) = v_0(s_2) = v_0(s_3) = v_0(s_4) = 0$ 。

将 $v_0(s_i)$ 代入表 4.1 可得表 4.2 所示 q 值。

表 4.2 $k = 0$ 时的 $q(s, a)$ 值

q -table	a_1	a_2	a_3	a_4	a_5
s_1	-1	-1	0	-1	0
s_2	-1	-1	1	0	-1
s_3	0	1	-1	-1	0
s_4	-1	-1	-1	0	1

策略更新: 对每个状态选择 q 值最大的动作, 得到 π_1 :

$$\pi_1(a_5 | s_1) = 1, \pi_1(a_3 | s_2) = 1, \pi_1(a_2 | s_3) = 1, \pi_1(a_5 | s_4) = 1$$

该策略不是最优的, 因为它在 s_1 选择了原地不动 (a_5)。注意 (s_1, a_5) 和 (s_1, a_3) 的 q 值相同, 可以任意选择。

价值更新: 将 v 值更新为各状态的最大 q 值:

$$v_1(s_1) = 0, v_1(s_2) = 1, v_1(s_3) = 1, v_1(s_4) = 1$$

$k = 1$: 将 $v_1(s_i)$ 代入表 4.1 可得表 4.3 所示 q 值。

表 4.3 $k = 1$ 时的 $q(s, a)$ 值

q -table	a_1	a_2	a_3	a_4	a_5
s_1	$-1 + \gamma \cdot 0$	$-1 + \gamma \cdot 1$	$0 + \gamma \cdot 1$	$-1 + \gamma \cdot 0$	$0 + \gamma \cdot 0$
s_2	$-1 + \gamma \cdot 1$	$-1 + \gamma \cdot 1$	$1 + \gamma \cdot 1$	$0 + \gamma \cdot 0$	$-1 + \gamma \cdot 1$
s_3	$0 + \gamma \cdot 0$	$1 + \gamma \cdot 1$	$-1 + \gamma \cdot 1$	$-1 + \gamma \cdot 1$	$0 + \gamma \cdot 1$
s_4	$-1 + \gamma \cdot 1$	$-1 + \gamma \cdot 1$	$-1 + \gamma \cdot 1$	$0 + \gamma \cdot 1$	$1 + \gamma \cdot 1$

策略更新: $\pi_2(a_3 | s_1) = 1, \pi_2(a_3 | s_2) = 1, \pi_2(a_2 | s_3) = 1, \pi_2(a_5 | s_4) = 1$ 。

价值更新:

$$v_2(s_1) = \gamma \cdot 1, v_2(s_2) = 1 + \gamma \cdot 1, v_2(s_3) = 1 + \gamma \cdot 1, v_2(s_4) = 1 + \gamma \cdot 1$$

$k = 2, 3, 4, \dots$: 值得注意的是, π_2 已经是最优策略。因此在这个简单示例中, 只需两次迭代即可获得最优策略。对于更复杂的示例, 需要运行更多次迭代, 直到 v_k 收敛 (例如直到 $\|v_{k+1} - v_k\|$ 小于预设阈值)。

4.2 策略迭代

本节介绍另一个重要算法: **策略迭代 (Policy Iteration)**。与值迭代不同, 策略迭代并非直接求解贝尔曼最优方程, 但它与值迭代有着密切的联系。此外, 策略迭代的基本思想在强化学习算法中得到了广泛应用, 因此非常重要。

4.2.1 算法分析

策略迭代是一种迭代算法, 每次迭代包含两个步骤:

- **策略评估步骤 (Policy Evaluation):** 求解给定策略 π_k 的状态价值，即求解以下贝尔曼方程：

$$\mathbf{v}_{\pi_k} = \mathbf{r}_{\pi_k} + \gamma \mathbf{P}_{\pi_k} \mathbf{v}_{\pi_k} \quad (3)$$

其中 π_k 是上一次迭代得到的策略， $\mathbf{v}_{\pi_k}$ 是待计算的状态价值。 $\mathbf{r}_{\pi_k}$ 和 $\mathbf{P}_{\pi_k}$ 可从系统模型中获得。

- **策略改进步骤 (Policy Improvement):** 利用第一步计算出的 $\mathbf{v}_{\pi_k}$ ，通过以下方式得到新策略：

$$\pi_{k+1} = \arg \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_{\pi_k}) \quad (4)$$

上述算法描述自然地引出以下三个问题：

1. 策略评估步骤中，如何求解 $\mathbf{v}_{\pi_k}$ ？
2. 策略改进步骤中，为什么新策略 π_{k+1} 优于 π_k ？
3. 为什么该算法最终能收敛到最优策略？

如何计算 $\mathbf{v}_{\pi_k}$ ？

第 2 讲介绍了两种求解贝尔曼方程 (3) 的方法：

- **闭式解:** $\mathbf{v}_{\pi_k} = (\mathbf{I} - \gamma \mathbf{P}_{\pi_k})^{-1} \mathbf{r}_{\pi_k}$ 。该方法便于理论分析，但由于需要计算矩阵逆，实际实现效率较低。
- **迭代算法:**

$$\mathbf{v}_{\pi_k}^{(j+1)} = \mathbf{r}_{\pi_k} + \gamma \mathbf{P}_{\pi_k} \mathbf{v}_{\pi_k}^{(j)}, \quad j = 0, 1, 2, \dots \quad (5)$$

其中 $\mathbf{v}_{\pi_k}^{(j)}$ 表示 $\mathbf{v}_{\pi_k}$ 的第 j 次估计。从任意初始猜测 $\mathbf{v}_{\pi_k}^{(0)}$ 出发，保证 $\mathbf{v}_{\pi_k}^{(j)} \rightarrow \mathbf{v}_{\pi_k}$ 当 $j \rightarrow \infty$ 。

有趣的是，策略迭代本身就是一个迭代算法，而其策略评估步骤中又嵌入了另一个迭代算法 (5)。理论上，该嵌入的迭代算法需要无穷多步（即 $j \rightarrow \infty$ ）才能收敛到真实状态价值 $\mathbf{v}_{\pi_k}$ 。实际上，当满足某个终止条件时迭代即可停止，例如 $\|\mathbf{v}_{\pi_k}^{(j+1)} - \mathbf{v}_{\pi_k}^{(j)}\|$ 小于预设阈值，或 j 超过预设值。如果我们不运行无穷步迭代，就只能得到 $\mathbf{v}_{\pi_k}$ 的不精确值，这会造成问题吗？答案是不会——原因将在第 4.3 节介绍截断策略迭代时说明。

为什么 π_{k+1} 优于 π_k ？

定理 4.1 策略改进

若 $\pi_{k+1} = \arg \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_{\pi_k})$ ，则 $\mathbf{v}_{\pi_{k+1}} \geq \mathbf{v}_{\pi_k}$ 。这里 $\mathbf{v}_{\pi_{k+1}} \geq \mathbf{v}_{\pi_k}$ 表示对所有 s 都有 $v_{\pi_{k+1}}(s) \geq v_{\pi_k}(s)$ 。

证明

由于 $\mathbf{v}_{\pi_{k+1}}$ 和 $\mathbf{v}_{\pi_k}$ 都是状态价值，它们满足各自的贝尔曼方程：

$$\mathbf{v}_{\pi_{k+1}} = \mathbf{r}_{\pi_{k+1}} + \gamma \mathbf{P}_{\pi_{k+1}} \mathbf{v}_{\pi_{k+1}}, \quad \mathbf{v}_{\pi_k} = \mathbf{r}_{\pi_k} + \gamma \mathbf{P}_{\pi_k} \mathbf{v}_{\pi_k}$$

由于 $\pi_{k+1} = \arg \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_{\pi_k})$ ，可知：

$$\mathbf{r}_{\pi_{k+1}} + \gamma \mathbf{P}_{\pi_{k+1}} \mathbf{v}_{\pi_k} \geq \mathbf{r}_{\pi_k} + \gamma \mathbf{P}_{\pi_k} \mathbf{v}_{\pi_k}$$

由此可得：

$$\begin{aligned} \mathbf{v}_{\pi_k} - \mathbf{v}_{\pi_{k+1}} &= (\mathbf{r}_{\pi_k} + \gamma \mathbf{P}_{\pi_k} \mathbf{v}_{\pi_k}) - (\mathbf{r}_{\pi_{k+1}} + \gamma \mathbf{P}_{\pi_{k+1}} \mathbf{v}_{\pi_{k+1}}) \\ &\leq (\mathbf{r}_{\pi_{k+1}} + \gamma \mathbf{P}_{\pi_{k+1}} \mathbf{v}_{\pi_k}) - (\mathbf{r}_{\pi_{k+1}} + \gamma \mathbf{P}_{\pi_{k+1}} \mathbf{v}_{\pi_{k+1}}) \\ &= \gamma \mathbf{P}_{\pi_{k+1}} (\mathbf{v}_{\pi_k} - \mathbf{v}_{\pi_{k+1}}) \end{aligned}$$

因此:

$$\mathbf{v}_{\pi_k} - \mathbf{v}_{\pi_{k+1}} \leq \gamma^n \mathbf{P}_{\pi_{k+1}}^n (\mathbf{v}_{\pi_k} - \mathbf{v}_{\pi_{k+1}}) \leq \lim_{n \rightarrow \infty} \gamma^n \mathbf{P}_{\pi_{k+1}}^n (\mathbf{v}_{\pi_k} - \mathbf{v}_{\pi_{k+1}}) = 0$$

极限为零是因为 $\gamma^n \rightarrow 0$ (当 $n \rightarrow \infty$) 且 $\mathbf{P}_{\pi_{k+1}}^n$ 对任意 n 都是非负随机矩阵。故 $\mathbf{v}_{\pi_{k+1}} \geq \mathbf{v}_{\pi_k}$ 。

为什么策略迭代算法最终能找到最优策略?

策略迭代算法产生两个序列:策略序列 $\{\pi_0, \pi_1, \dots, \pi_k, \dots\}$ 和状态价值序列 $\{\mathbf{v}_{\pi_0}, \mathbf{v}_{\pi_1}, \dots, \mathbf{v}_{\pi_k}, \dots\}$ 。设 $\mathbf{v}^*$ 为最优状态价值, 则 $\mathbf{v}_{\pi_k} \leq \mathbf{v}^*$ 对所有 k 成立。由引理 4.1, 策略不断改进, 故:

$$\mathbf{v}_{\pi_0} \leq \mathbf{v}_{\pi_1} \leq \mathbf{v}_{\pi_2} \leq \dots \leq \mathbf{v}_{\pi_k} \leq \dots \leq \mathbf{v}^*$$

由于 $\mathbf{v}_{\pi_k}$ 单调不减且有上界 $\mathbf{v}^*$, 由单调收敛定理可知 $\mathbf{v}_{\pi_k}$ 收敛到某个常数值, 记为 $\mathbf{v}_\infty$ (当 $k \rightarrow \infty$)。下面的定理表明 $\mathbf{v}_\infty = \mathbf{v}^*$ 。

定理 4.2 策略迭代的收敛性

策略迭代算法产生的状态价值序列 $\{\mathbf{v}_{\pi_k}\}_{k=0}^\infty$ 收敛到最优状态价值 $\mathbf{v}^*$ 。因此, 策略序列 $\{\pi_k\}_{k=0}^\infty$ 收敛到最优策略。

证明

证明的核心思路是展示策略迭代算法的收敛速度比值迭代算法更快。

具体来说, 为证明 $\{\mathbf{v}_{\pi_k}\}_{k=0}^\infty$ 的收敛性, 我们引入另一个序列 $\{\mathbf{v}_k\}_{k=0}^\infty$, 由下式生成:

$$\mathbf{v}_{k+1} = f(\mathbf{v}_k) = \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_k)$$

这正是值迭代算法, 我们已知对任意初值 $\mathbf{v}_0$, $\mathbf{v}_k$ 都收敛到 $\mathbf{v}^*$ 。

对 $k=0$, 总可以找到 $\mathbf{v}_0$ 使得对任意 π_0 有 $\mathbf{v}_{\pi_0} \geq \mathbf{v}_0$ 。

下面用归纳法证明对所有 k 都有 $\mathbf{v}_k \leq \mathbf{v}_{\pi_k} \leq \mathbf{v}^*$ 。

对 $k \geq 0$, 假设 $\mathbf{v}_{\pi_k} \geq \mathbf{v}_k$ 。则对 $k+1$ 有:

$$\begin{aligned} \mathbf{v}_{\pi_{k+1}} - \mathbf{v}_{k+1} &= (\mathbf{r}_{\pi_{k+1}} + \gamma \mathbf{P}_{\pi_{k+1}} \mathbf{v}_{\pi_{k+1}}) - \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_k) \\ &\geq (\mathbf{r}_{\pi_{k+1}} + \gamma \mathbf{P}_{\pi_{k+1}} \mathbf{v}_{\pi_k}) - \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_k) \quad (\text{由引理 4.1, } \mathbf{v}_{\pi_{k+1}} \geq \mathbf{v}_{\pi_k}) \\ &= (\mathbf{r}_{\pi_{k+1}} + \gamma \mathbf{P}_{\pi_{k+1}} \mathbf{v}_{\pi_k}) - (\mathbf{r}_{\pi'_k} + \gamma \mathbf{P}_{\pi'_k} \mathbf{v}_k) \quad (\text{设 } \pi'_k = \arg \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_k)) \\ &\geq (\mathbf{r}_{\pi'_k} + \gamma \mathbf{P}_{\pi'_k} \mathbf{v}_{\pi_k}) - (\mathbf{r}_{\pi'_k} + \gamma \mathbf{P}_{\pi'_k} \mathbf{v}_k) \quad (\text{因 } \pi_{k+1} = \arg \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_{\pi_k})) \\ &= \gamma \mathbf{P}_{\pi'_k} (\mathbf{v}_{\pi_k} - \mathbf{v}_k) \end{aligned}$$

由于 $\mathbf{v}_{\pi_k} - \mathbf{v}_k \geq 0$ 且 $\mathbf{P}_{\pi'_k}$ 是非负的, 有 $\mathbf{P}_{\pi'_k} (\mathbf{v}_{\pi_k} - \mathbf{v}_k) \geq 0$, 因此 $\mathbf{v}_{\pi_{k+1}} - \mathbf{v}_{k+1} \geq 0$ 。

由归纳法可知对所有 $k \geq 0$ 都有 $\mathbf{v}_k \leq \mathbf{v}_{\pi_k} \leq \mathbf{v}^*$ 。由于 $\mathbf{v}_k$ 收敛到 $\mathbf{v}^*$, 由夹逼原理, $\mathbf{v}_{\pi_k}$ 也收敛到 $\mathbf{v}^*$ 。

该证明不仅展示了策略迭代的收敛性, 还揭示了策略迭代与值迭代之间的关系: 如果两个算法从相同的初始条件出发, 由于策略评估步骤中嵌入了额外的迭代, 策略迭代的收敛速度将快于值迭代。这一点在第 4.3 节介绍截断策略迭代时将更加清晰。

4.2.2 元素形式与实现

为实现策略迭代算法, 需要分析其元素形式。

策略评估步骤：使用迭代算法 (5) 求解 v_{π_k} 。其元素形式为：

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a | s) \left(\sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_{\pi_k}^{(j)}(s') \right), \quad s \in \mathcal{S}$$

其中 $j = 0, 1, 2, \dots$ 。

策略改进步骤：求解 $\pi_{k+1} = \arg \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_{\pi_k})$ 。其元素形式为：

$$\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a | s) \underbrace{\left(\sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_{\pi_k}(s') \right)}_{q_{\pi_k}(s, a)}, \quad s \in \mathcal{S}$$

其中 $q_{\pi_k}(s, a)$ 是策略 π_k 下的动作价值。设 $a_k^*(s) = \arg \max_a q_{\pi_k}(s, a)$ ，则贪心最优策略为：

$$\pi_{k+1}(a | s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & a \neq a_k^*(s) \end{cases}$$

算法 4.2 策略迭代算法

初始化：已知所有 (s, a) 的系统模型 $p(r | s, a)$ 和 $p(s' | s, a)$ 。初始猜测 π_0 。

目标：求解最优状态价值与最优策略。

当 v_{π_k} 未收敛，对第 k 次迭代执行：

- **策略评估：**

- 初始化：任意初始猜测 $v_{\pi_k}^{(0)}$
- 当 $v_{\pi_k}^{(j)}$ 未收敛，对第 j 次迭代执行：
 - * 对每个状态 $s \in \mathcal{S}$ ： $v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a | s) [\sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_{\pi_k}^{(j)}(s')]$

- **策略改进：**

- 对每个状态 $s \in \mathcal{S}$ ，执行：
 - * 对每个动作 $a \in \mathcal{A}$ ： $q_{\pi_k}(s, a) = \sum_r p(r | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_{\pi_k}(s')$
 - * $a_k^*(s) = \arg \max_a q_{\pi_k}(s, a)$
 - * $\pi_{k+1}(a | s) = 1$ 若 $a = a_k^*$ ，否则为 0

4.2.3 示例

简单示例

考虑一个简单示例（如图 4.3 所示）：两个状态 $\{s_1, s_2\}$ ，三个动作 $\mathcal{A} = \{a_l, a_0, a_r\}$ ，分别表示左移、不动和右移。奖励设置 $r_{\text{boundary}} = -1$ ， $r_{\text{target}} = 1$ ， $\gamma = 0.9$ 。

从初始策略出发（在 s_1 处选择左移， s_2 处选择不动），该策略并不好，无法到达目标区域。

$k = 0$ ，策略评估步骤：求解贝尔曼方程

$$v_{\pi_0}(s_1) = -1 + \gamma v_{\pi_0}(s_1), \quad v_{\pi_0}(s_2) = 0 + \gamma v_{\pi_0}(s_1)$$

手动解得 $v_{\pi_0}(s_1) = -10$ ， $v_{\pi_0}(s_2) = -9$ 。

在实践中，可用迭代算法求解。例如取初值 $v_{\pi_0}^{(0)}(s_1) = v_{\pi_0}^{(0)}(s_2) = 0$ ，则：

$$\begin{aligned} v_{\pi_0}^{(1)}(s_1) &= -1 + \gamma \cdot 0 = -1, & v_{\pi_0}^{(1)}(s_2) &= 0 + \gamma \cdot 0 = 0 \\ v_{\pi_0}^{(2)}(s_1) &= -1 + \gamma \cdot (-1) = -1.9, & v_{\pi_0}^{(2)}(s_2) &= 0 + \gamma \cdot (-1) = -0.9 \\ v_{\pi_0}^{(3)}(s_1) &= -1 + \gamma \cdot (-1.9) = -2.71, & v_{\pi_0}^{(3)}(s_2) &= 0 + \gamma \cdot (-1.9) = -1.71 \\ &\dots & & \end{aligned}$$

随着 j 增大, $v_{\pi_0}^{(j)}(s_1) \rightarrow -10$, $v_{\pi_0}^{(j)}(s_2) \rightarrow -9$ 。

策略改进步骤: 关键计算每个状态-动作对的 $q_{\pi_0}(s, a)$ 。

表 4.4 $q_{\pi_k}(s, a)$ 的表达式

$q_{\pi_k}(s, a)$	a_ℓ	a_0	a_r
s_1	$-1 + \gamma v_{\pi_k}(s_1)$	$0 + \gamma v_{\pi_k}(s_1)$	$1 + \gamma v_{\pi_k}(s_2)$
s_2	$0 + \gamma v_{\pi_k}(s_1)$	$1 + \gamma v_{\pi_k}(s_2)$	$-1 + \gamma v_{\pi_k}(s_2)$

代入 $v_{\pi_0}(s_1) = -10$, $v_{\pi_0}(s_2) = -9$:

表 4.5 $k = 0$ 时的 $q_{\pi_0}(s, a)$ 值

$q_{\pi_0}(s, a)$	a_ℓ	a_0	a_r
s_1	-10	-9	-7.1
s_2	-9	-7.1	-9.1

取 q_{π_0} 中最大的值, 得到改进后的策略 π_1 :

$$\pi_1(a_r | s_1) = 1, \quad \pi_1(a_0 | s_2) = 1$$

该策略是最优策略。在该简单示例中, 仅需一次迭代即可找到最优策略。

更复杂的示例

考虑一个 5×5 的网格世界 (如图 4.4 所示)。奖励设置 $r_{\text{boundary}} = -1$, $r_{\text{forbidden}} = -10$, $r_{\text{target}} = 1$, $\gamma = 0.9$ 。从随机初始策略出发, 策略迭代算法最终收敛到最优策略。

在迭代过程中可观察到两个有趣现象:

1. 靠近目标区域的状态先于远离的状态找到最优策略。只有当近距离状态先找到通往目标的路径, 远距离状态才能找到经过近状态到达目标的路径。
2. 状态价值的空间分布呈现出「离目标越近价值越高」的模式。这是因为从更远的初始状态出发, 智能体需要更多步才能获得正奖励, 这些奖励会经过严重的折扣衰减。

4.3 截断策略迭代

本节介绍一个更一般的算法, 称为**截断策略迭代 (Truncated Policy Iteration)**。我们将看到, 值迭代和策略迭代实际上是截断策略迭代的两个极端特例。

4.3.1 值迭代与策略迭代的对比

首先对比两种算法的步骤:

策略迭代: 选择任意初始策略 π_0 。在第 k 次迭代中:

- 步骤 1 (PE): 给定 π_k , 从 $\mathbf{v}_{\pi_k} = \mathbf{r} + \gamma \mathbf{P}_{\pi_k} \mathbf{v}_{\pi_k}$ 求解 $\mathbf{v}_{\pi_k}$
- 步骤 2 (PI): 给定 $\mathbf{v}_{\pi_k}$, 从 $\pi_{k+1} = \arg \max_{\pi} (\mathbf{r} + \gamma \mathbf{P}_{\pi} \mathbf{v}_{\pi_k})$ 求解 π_{k+1}

值迭代: 选择任意初始价值 $\mathbf{v}_0$ 。在第 k 次迭代中:

- 步骤 1 (PU): 给定 $\mathbf{v}_k$, 从 $\pi_{k+1} = \arg \max_{\pi} (\mathbf{r} + \gamma \mathbf{P}_{\pi} \mathbf{v}_k)$ 求解 π_{k+1}
- 步骤 2 (VU): 给定 π_{k+1} , 从 $\mathbf{v}_{k+1} = \mathbf{r} + \gamma \mathbf{P}_{\pi_{k+1}} \mathbf{v}_k$ 求解 $\mathbf{v}_{k+1}$

这两种算法的流程可以表示为:

$$\begin{aligned} \text{策略迭代: } \pi_0 &\xrightarrow{\text{PE}} \mathbf{v}_{\pi_0} \xrightarrow{\text{PI}} \pi_1 \xrightarrow{\text{PE}} \mathbf{v}_{\pi_1} \xrightarrow{\text{PI}} \pi_2 \xrightarrow{\text{PE}} \mathbf{v}_{\pi_2} \xrightarrow{\text{PI}} \dots \\ \text{值迭代: } \mathbf{v}_0 &\xrightarrow{\text{PU}} \pi_1 \xrightarrow{\text{VU}} \mathbf{v}_1 \xrightarrow{\text{PU}} \pi_2 \xrightarrow{\text{VU}} \mathbf{v}_2 \xrightarrow{\text{PU}} \dots \end{aligned}$$

为了看清两者的差异，令两种算法从相同的初始条件出发： $\mathbf{v}_0 = \mathbf{v}_{\pi_0}$ 。表 4.6 对比了两种算法的执行步骤。

表 4.6 策略迭代与值迭代的执行步骤对比

步骤	策略迭代	值迭代	备注
1)策略	π_0	——	
2)价值	$\mathbf{v}_{\pi_0} = \mathbf{r}_{\pi_0} + \gamma \mathbf{P}_{\pi_0} \mathbf{v}_{\pi_0}$	$\mathbf{v}_0 := \mathbf{v}_{\pi_0}$	
3)策略	$\pi_1 = \arg \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_{\pi_0})$	$\pi_1 = \arg \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_0)$	两个策略相同 $\mathbf{v}_{\pi_1} \geq \mathbf{v}_1$
4)价值	$\mathbf{v}_{\pi_1} = \mathbf{r}_{\pi_1} + \gamma \mathbf{P}_{\pi_1} \mathbf{v}_{\pi_1}$	$\mathbf{v}_1 = \mathbf{r}_{\pi_1} + \gamma \mathbf{P}_{\pi_1} \mathbf{v}_0$	
5)策略	$\pi_2 = \arg \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_{\pi_1})$	$\pi'_2 = \arg \max_{\pi} (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_1)$	
...	...	...	

前三个步骤中，两种算法产生相同结果（因为 $\mathbf{v}_0 = \mathbf{v}_{\pi_0}$ ）。从第四步开始，差异出现：值迭代执行一步计算 $\mathbf{v}_1 = \mathbf{r}_{\pi_1} + \gamma \mathbf{P}_{\pi_1} \mathbf{v}_0$ ，而策略迭代求解 $\mathbf{v}_{\pi_1} = \mathbf{r}_{\pi_1} + \gamma \mathbf{P}_{\pi_1} \mathbf{v}_{\pi_1}$ ，这需要无穷多步迭代。

将策略迭代第四步中求解 $\mathbf{v}_{\pi_1}$ 的迭代过程显式写出（令 $\mathbf{v}_{\pi_1}^{(0)} = \mathbf{v}_0$ ）：

$$\begin{aligned}
 & \mathbf{v}_{\pi_1}^{(0)} = \mathbf{v}_0 \\
 & \text{值迭代: } \mathbf{v}_1 \quad \text{——} \quad \mathbf{v}_{\pi_1}^{(1)} = \mathbf{r}_{\pi_1} + \gamma \mathbf{P}_{\pi_1} \mathbf{v}_{\pi_1}^{(0)} \\
 & \quad \quad \quad \text{——} \quad \mathbf{v}_{\pi_1}^{(2)} = \mathbf{r}_{\pi_1} + \gamma \mathbf{P}_{\pi_1} \mathbf{v}_{\pi_1}^{(1)} \\
 & \quad \quad \quad \dots \\
 & \text{截断策略迭代: } \tilde{\mathbf{v}}_1 \quad \text{——} \quad \mathbf{v}_{\pi_1}^{(j)} = \mathbf{r}_{\pi_1} + \gamma \mathbf{P}_{\pi_1} \mathbf{v}_{\pi_1}^{(j-1)} \\
 & \quad \quad \quad \dots \\
 & \text{策略迭代: } \mathbf{v}_{\pi_1} \quad \text{——} \quad \mathbf{v}_{\pi_1}^{(\infty)} = \mathbf{r}_{\pi_1} + \gamma \mathbf{P}_{\pi_1} \mathbf{v}_{\pi_1}^{(\infty)}
 \end{aligned}$$

由此可以得到以下观察：

- 若策略评估步骤仅执行一次迭代，则 $\mathbf{v}_{\pi_1}^{(1)}$ 实际上就是值迭代中的 $\mathbf{v}_1$ 。
- 若执行无穷多次迭代，则 $\mathbf{v}_{\pi_1}^{(\infty)}$ 就是策略迭代中的 $\mathbf{v}_{\pi_1}$ 。
- 若执行有限 j_{truncate} 次迭代，则该算法称为**截断策略迭代**。之所以称为「截断」，是因为从 j_{truncate} 到 ∞ 的迭代被截去了。

因此，值迭代和策略迭代可以看作截断策略迭代的两个极端情况：值迭代在 $j_{\text{truncate}} = 1$ 处终止，策略迭代在 $j_{\text{truncate}} = \infty$ 处终止。

4.3.2 截断策略迭代算法

简而言之，截断策略迭代与策略迭代的唯一区别在于策略评估步骤仅执行有限步迭代。其实现细节总结在算法 4.3 中。

算法 4.3 截断策略迭代算法

初始化：已知所有 (s, a) 的概率模型 $p(r | s, a)$ 和 $p(s' | s, a)$ 。初始猜测 π_0 。

目标：求解最优状态价值与最优策略。

当 v_k 未收敛，对第 k 次迭代执行：

- 策略评估：

- 初始化：选取初始猜测 $v_k^{(0)} = v_{k-1}$ 。设置最大迭代次数 j_{truncate} 。
- 当 $j < j_{\text{truncate}}$ ，执行：
 - * 对每个状态 $s \in \mathcal{S}$ ： $v_k^{(j+1)}(s) = \sum_a \pi_k(a | s) [\sum_r p(r | s, a)r + \gamma \sum_{s'} p(s' | s, a)v_k^{(j)}(s')]$
- 设置 $v_k = v_k^{(j_{\text{truncate}})}$

- 策略改进：

- 对每个状态 $s \in \mathcal{S}$ ，执行：
 - * 对每个动作 $a \in \mathcal{A}(s)$ ： $q_k(s, a) = \sum_r p(r | s, a)r + \gamma \sum_{s'} p(s' | s, a)v_k(s')$
 - * $a_k^*(s) = \arg \max_a q_k(s, a)$
 - * $\pi_{k+1}(a | s) = 1$ 若 $a = a_k^*$ ，否则为 0

备注

算法中的 v_k 和 $v_k^{(j)}$ 并非真实的状态价值，而是真实状态价值的近似值，因为策略评估步骤只执行了有限次迭代。

即使 v_k 不等于 v_{π_k} ，算法仍然能够找到最优策略。直观上，截断策略迭代介于值迭代和策略迭代之间：一方面，由于在策略评估步骤中执行了多于一次的迭代，它比值迭代收敛得更快；另一方面，由于仅执行有限次迭代，它比策略迭代的每次迭代计算量更小。三者之间的关系如图 4.5 所示。

定理 4.3 价值改进

考虑策略评估步骤中的迭代算法：

$$v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \quad j = 0, 1, 2, \dots$$

若初始猜测选为 $v_{\pi_k}^{(0)} = v_{\pi_{k-1}}$ ，则对 $j = 0, 1, 2, \dots$ 有

$$v_{\pi_k}^{(j+1)} \geq v_{\pi_k}^{(j)}$$

证明

首先，由于 $v_{\pi_k}^{(j)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j-1)}$ 且 $v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}$ ，有：

$$v_{\pi_k}^{(j+1)} - v_{\pi_k}^{(j)} = \gamma P_{\pi_k} (v_{\pi_k}^{(j)} - v_{\pi_k}^{(j-1)}) = \dots = \gamma^j P_{\pi_k}^j (v_{\pi_k}^{(1)} - v_{\pi_k}^{(0)})$$

其次，由于 $v_{\pi_k}^{(0)} = v_{\pi_{k-1}}$ ，有：

$$\begin{aligned} v_{\pi_k}^{(1)} &= r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(0)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_{k-1}} \\ &\geq r_{\pi_{k-1}} + \gamma P_{\pi_{k-1}} v_{\pi_{k-1}} = v_{\pi_{k-1}} = v_{\pi_k}^{(0)} \end{aligned}$$

其中不等号是因为 $\pi_k = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_{k-1}})$ 。

将 $v_{\pi_k}^{(1)} \geq v_{\pi_k}^{(0)}$ 代入可得 $v_{\pi_k}^{(j+1)} \geq v_{\pi_k}^{(j)}$ 。

值得注意的是，命题 4.3 要求假设 $\mathbf{v}_{\pi_k}^{(0)} = \mathbf{v}_{\pi_{k-1}}$ 。然而在实际中， $\mathbf{v}_{\pi_{k-1}}$ 是不可获得的，只有 $\mathbf{v}_{k-1}$ 是可用的。尽管如此，该命题仍为截断策略迭代的收敛性提供了洞见。

截断策略迭代的优势：与策略迭代相比，截断策略迭代在策略评估步骤中只需执行有限步迭代，因此计算效率更高；与值迭代相比，截断策略迭代通过在策略评估步骤中多执行几步迭代，可以加速整体收敛。

4.4 本章小结

本章介绍了三种可用于求解最优策略的算法：

- **值迭代：**正是压缩映射定理所建议的求解贝尔曼最优方程的算法。每次迭代可分为价值更新和策略更新两个步骤。值迭代的中间值 v_k 不是任何策略的状态价值。
- **策略迭代：**比值迭代稍复杂，每次迭代包含策略评估和策略改进两个步骤。策略评估步骤中嵌入了求解贝尔曼方程的迭代过程。策略迭代的中间值 $\mathbf{v}_{\pi_k}$ 是真实的状态价值。
- **截断策略迭代：**值迭代和策略迭代可看作截断策略迭代的两个极端特例（分别对应 $j_{\text{truncate}} = 1$ 和 $j_{\text{truncate}} = \infty$ ）。

这三种算法的一个共同特点是每次迭代都包含两个步骤：一个步骤更新价值，另一个步骤更新策略。价值与策略交互更新的思想广泛存在于强化学习算法中，这一思想也被称为**广义策略迭代 (Generalized Policy Iteration)**。

最后，本章介绍的算法都需要系统模型。从第 5 讲开始，我们将研究无模型强化学习算法。我们将看到，无模型算法可以通过推广本章介绍的算法得到。

备注

本章算法需要已知 $p(r | s, a)$ 和 $p(s' | s, a)$ ，通常被称为**动态规划算法**而非强化学习算法。强化学习算法可分为**基于模型 (model-based)**和**无模型 (model-free)**两类。这里的「基于模型」并非指需要系统模型，而是指利用数据估计系统模型并在学习过程中使用该模型。无模型强化学习则在学习过程中不涉及模型估计。

4.5 常见问题解答

- **Q：值迭代算法一定能找到最优策略吗？**
A：是的。值迭代正是第 3 讲中压缩映射定理所建议的求解贝尔曼最优方程的算法，该算法的收敛性由压缩映射定理保证。
- **Q：值迭代产生的中间价值是状态价值吗？**
A：不是。这些中间值并不保证满足任何策略的贝尔曼方程。它们只是算法生成的中间值。
- **Q：策略迭代算法包含哪些步骤？**
A：策略迭代的每次迭代包含两个步骤：策略评估和策略改进。策略评估步骤求解贝尔曼方程以获得当前策略的状态价值；策略改进步骤更新策略，使新策略具有更大的状态价值。
- **Q：策略迭代算法中是否嵌入了另一个迭代算法？**
A：是的。在策略迭代的策略评估步骤中，需要用一个迭代算法来求解当前策略的贝尔曼方程。
- **Q：策略迭代产生的中间价值是状态价值吗？**
A：是的。因为这些值正是当前策略的贝尔曼方程的解。
- **Q：策略迭代算法一定能找到最优策略吗？**
A：是的。本章给出了其收敛性的严格证明：策略价值序列单调不减且有上界，由单调收敛定理和夹逼原理可证其收敛到最优状态价值。
- **Q：截断策略迭代与策略迭代的关系是什么？**

A: 顾名思义, 截断策略迭代可以从策略迭代出发, 在策略评估步骤仅执行有限步迭代而得到。它是策略迭代的计算高效版本。

- **Q: 截断策略迭代与值迭代的关系是什么?**

A: 值迭代可以看作截断策略迭代的极端情况——在策略评估步骤中仅执行一步迭代。

- **Q: 截断策略迭代产生的中间价值是状态价值吗?**

A: 不是。只有在策略评估步骤中执行无穷步迭代, 才能得到真实状态价值。执行有限步迭代只能得到真实状态价值的近似。

- **Q: 在截断策略迭代的策略评估步骤中应执行多少步迭代?**

A: 一般建议是执行几步但不至于太多。在策略评估步骤中多执行几步迭代可以加速整体收敛速度, 但执行太多步则不会显著进一步加速收敛。

- **Q: 什么是广义策略迭代?**

A: 广义策略迭代并非特指某个具体算法, 而是指价值与策略交替更新的通用思想。这一思想根植于策略迭代算法, 本书介绍的大多数强化学习算法都属于广义策略迭代的范畴。

- **Q: 为什么需要截断策略迭代?**

A: 策略迭代的策略评估步骤要求解贝尔曼方程, 这需要理论上的无穷步迭代, 计算量很大。截断策略迭代通过控制策略评估步数, 在计算量和收敛速度之间取得平衡, 是实际应用中最实用的形式。